

Task 6.2C - My DNS Server

Introduction

This is basically the same code that I submitted in Task 5.2C.

Below is a screenshot of myself running this simple implementation:

<pre>PS C:\Users\uncus\Documents\GitHub\uni-deakin-26-t2\sit202_networking\task62c> python .\server.py DNS server is running on 127.0.0.1:5354 Query received: google.com - CNAME Query received: google.com - A Query received: api.example.com - A █</pre>	<pre>PS C:\Users\uncus\Documents\GitHub\uni-deakin-26-t2\sit202_networking\task62c> python .\client.py Enter hostname: google.com Enter record type (A or CNAME): CNAME Server response: DNS record not found PS C:\Users\uncus\Documents\GitHub\uni-deakin-26-t2\sit202_networking\task62c> python .\client.py Enter hostname: google.com Enter record type (A or CNAME): A Server response: A record: 142.250.70.14 PS C:\Users\uncus\Documents\GitHub\uni-deakin-26-t2\sit202_networking\task62c> python .\client.py Enter hostname: api.example.com Enter record type (A or CNAME): A Server response: A record: 192.168.1.30</pre>
---	--

Source Code

The python code has been attached to the end of this document.

➔ Next Page

DNS Server & Client (Python Implementation)

```
1  # -----
2  # server.py
3  # -----
4
5  import socket
6
7  HOST = "127.0.0.1"
8  PORT = 5354
9
10 records = {
11     "example.com": {
12         "@": {
13             "A": "93.184.216.34"
14         },
15         "www": {
16             "CNAME": "@"
17         },
18         "mail": {
19             "A": "192.168.1.20"
20         },
21         "api": {
22             "A": "192.168.1.30"
23         },
24         "shop": {
25             "CNAME": "www"
26         }
27     },
28
29     "google.com": {
30         "@": {
31             "A": "142.250.70.14"
32         },
33         "www": {
34             "CNAME": "@"
35         },
36         "mail": {
37             "CNAME": "@"
38         }
39     }
40 }
41
42 server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
43 server_socket.bind((HOST, PORT))
44
45 print(f"DNS server is running on {HOST}:{PORT}")
46
47
48
49 while True:
50     data, client_address = server_socket.recvfrom(1024)
```

```
51
52     query = data.decode()
53     hostname, record_type = query.split(",")
54
55     hostname = hostname.lower().strip()
56     record_type = record_type.upper().strip()
57
58     print(f"Query received: {hostname} - {record_type}")
59
60     answer = None
61
62     for domain, domain_records in records.items():
63         if hostname == domain:
64             host_label = "@"
65         elif hostname.endswith(f".{domain}"):
66             host_label = hostname[:-(len(domain) + 1)] # exclude the . before the
domain
67         else:
68             continue
69
70         host_records = domain_records.get(host_label, {})
71         answer = host_records.get(record_type)
72         break
73
74     if answer is not None:
75         response = f"{record_type} record: {answer}"
76     else:
77         response = "DNS record not found"
78
79     server_socket.sendto(response.encode(), client_address)
80
81     # -----
82     # client.py
83     # -----
84     import socket
85
86     SERVER_HOST = "127.0.0.1"
87     SERVER_PORT = 5354
88
89     client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
90     client_socket.settimeout(5)
91
92     hostname = input("Enter hostname: ")
93     record_type = input("Enter record type (A or CNAME): ")
94
95     query = f"{hostname},{record_type}"
96
97     client_socket.sendto(
98         query.encode(),
99         (SERVER_HOST, SERVER_PORT)
100    )
101
102    try:
```

```
103     response, server_address = client_socket.recvfrom(1024)
104     print("Server response:", response.decode())
105
106 except socket.timeout:
107     print("The DNS server did not respond.")
108
109 client_socket.close()
110
111
112
113
```